AIAC Assignment-10.2

K.Lokesh **2303A51201,B21**

Task Description -1(Error Detection and Correction)

Task:
Use AI to analyze a Python script and correct all syntax and logical errors.

Sample Input Code:

```
def calculate_total(nums)
   sum = 0
   for n in nums
      sum += n
   return total
```

Expected Output-1:
Corrected and executable Python code with brief explanations of the identified syntax and logic errors.

```
Corrected Code:
def calculate_total(nums):
    sum = 0
    for n in nums:
        sum += n
    return sum

Explanations of Identified Errors:
1. Syntax Error: The function definition was missing a colon (:) at the end of the line. This is required to indicate the start of the function body.
2. Syntax Error: The for loop was missing a colon (:) at the end of the line. This is required to indicate the start of the loop body.
```

Task Description -2(Code Style Standardization)

Task:
Use AI to refactor Python code to comply with standard coding style guidelines.

Sample Input Code:

```
def findSum(a,b):return a+b

print(findSum(5,10))
```

Expected Output-2:
Well-structured, consistently formatted Python code following standard style conventions.

```
def find_sum(a, b):
    return a + b
print(find_sum(5, 10))
```

```
1. Function names should be in snake_case (e.g., `find_sum` instead of `findSum`).
2. Use spaces around operators for better readability (e.g., `a + b` instead of `a+b`).
```

Task Description -3(Code Clarity Improvement)

Task:
Use AI to improve code readability without changing its functionality.

Sample Input Code:

```
def f(x,y):
 return x-y*2
print(f(10,3))
```

Expected Output-3:
Python code rewritten with meaningful function and variable names, proper indentation, and improved clarity.

```
def calculate_difference(x, y):
    return x - (y * 2)
result = calculate_difference(10, 3)
print(result)
```

Task Description -4(Structural Refactoring)

Task:
Use AI to refactor repetitive code into reusable functions.

Sample Input Code:

```
print("Hello Ram")
print("Hello Sita")
print("Hello Ravi")
```

Expected Output-4:
Modular Python code using reusable functions to eliminate repetition.

```
# Refactored code using a reusable function
def greet(name):
    print(f"Hello {name}")
# List of names to greet
names = ["Ram", "Sita", "Ravi"]
# Greet each name using the reusable function
for name in names:
    greet(name)
```

Task Description -5(Efficiency Enhancement)

Task:
Use AI to optimize Python code for better performance.

Sample Input Code:

```
numbers = [ ]

for i in range(1, 500000):

    numbers.append(i * i)

print(len(numbers))
```

Expected Output-5:
Optimized Python code that achieves the same result with improved performance.

```
'''Optimized Code:'''
numbers = [i * i for i in range(1, 500000)]
print(len(numbers))
```